

Assignment 1: Text Processing Fundamentals using MapReduce

Valentin Tian, 12426893
Tobias Haider, 11833743
Sevket Emin Cengay, 12552926
Burak Kilic, 12551186
Hafizullah Zubair, 12352877

April 27, 2026

Abstract

This project presents a scalable MapReduce-based approach for identifying discriminative unigram terms across product categories in the Amazon Reviews dataset. The proposed pipeline preprocesses review text, computes document frequencies per category, applies the chi-square statistical test to measure term-category association, and ranks terms according to their discriminative power. The implementation uses `mrjob` on a Hadoop cluster to support distributed processing of large-scale textual data. For each product category, the top 75 most discriminative terms are selected and additionally merged into a single alphabetically ordered dictionary. The project demonstrates the use of distributed statistical feature selection for large-scale text processing and category-specific feature extraction.

Contents

1. Introduction	3
2. Problem Overview	3
3. Methodology and Approach	4
4. Conclusion	6
A. Output example for one product category	7
B. Full Dataset Execution Log	7

1. Introduction

In this project, the problem of identifying discriminative terms across product categories in large-scale textual data using the Amazon Reviews dataset¹ is addressed. The dataset consists of millions of user-generated reviews, each associated with a product category and is used as an example of high-volume and unstructured data that requires scalable processing techniques.

The primary objective of the project is to extract informative unigram terms that best distinguish between different product categories. To achieve this, the chi-square (χ^2) statistical test was employed to measure the degree of association between term occurrence and product category.

The solution is implemented using the MapReduce programming model. This allows to distribute the computation across multiple nodes efficiently while maintaining scalability. The pipeline consists of several stages: text preprocessing, aggregation of document frequencies, computation of category-level statistics, and final ranking of terms based on their chi-square scores.

Special attention is given to preprocessing steps such as tokenization, case normalization and stopword filtering to ensure the quality and consistency of the extracted features.

The final output is a ranked list of the top discriminative terms for each category, along with a merged list of selected terms across all categories.

2. Problem Overview

The main problem addressed in this assignment is the identification of discriminative unigram terms across product categories in a large-scale review corpus. The dataset consists of Amazon product reviews stored as JSON records, where each record contains textual review content and the corresponding product category. The reviewText field represents the main textual input to be processed, while the category field provides the class label used for category-based term analysis.

A simple frequency-based analysis is not sufficient for this task, because frequently occurring terms are not necessarily useful for distinguishing one category from another. A term that appears often across all categories may have limited classification value. Therefore, the objective is to calculate chi-square scores for unigram terms in relation to each product category. Terms with higher chi-square values are considered more category-specific and more useful for feature selection in text classification.

The assignment requires selecting the top 75 most discriminative terms for each product category and then merging these selected terms into a single dictionary ordered alphabetically. This makes the task both a text processing problem and a feature selection problem. The resulting terms can be interpreted as category-specific indicators that may support downstream machine learning tasks such as document classification or category prediction.

Another important aspect of the problem is scalability. The final dataset is approximately 56GB in size and must be processed on a Hadoop cluster using MapReduce. This means that the solution must be designed to avoid unnecessary data replication, excessive intermediate output, and inefficient global computations. The preprocessing pipeline must also be consistent and deterministic: reviews need to be tokenized into unigrams, converted to lowercase, filtered using the provided stopwords list, and cleaned from one-character tokens.

¹<https://amazon-reviews-2023.github.io/>

Overall, the challenge is to design an efficient distributed text-processing pipeline that correctly computes category-level chi-square statistics and produces a compact, ordered output of the most discriminative terms per category.

3. Methodology and Approach

The proposed solution follows a two-stage MapReduce pipeline designed for scalable term discrimination analysis on large-scale review data. Both stages were implemented using the `mrjob` framework and executed on the Hadoop cluster. The first stage performs text preprocessing and document-frequency counting, while the second stage computes chi-square scores and selects the most discriminative terms per category. The overall MapReduce workflow used in the proposed solution is illustrated in Figure 1.

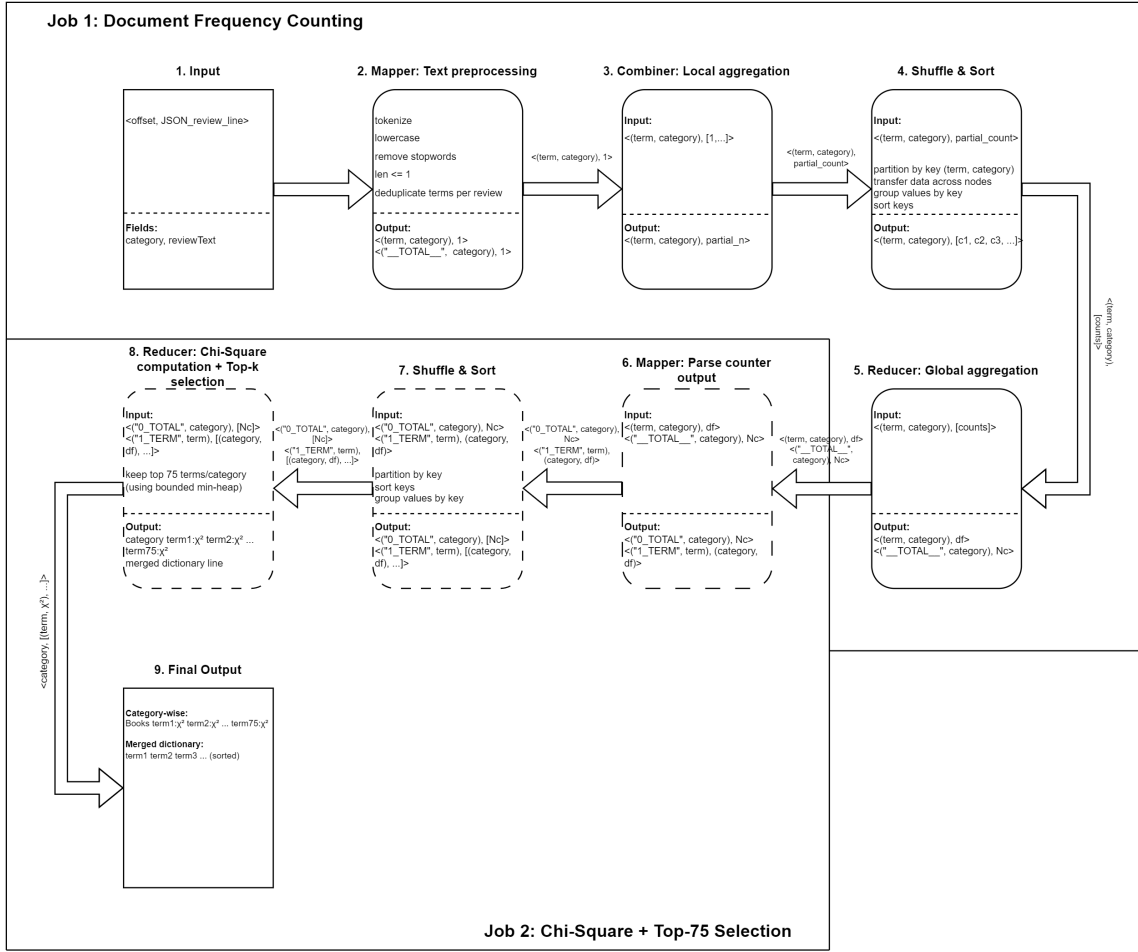


Figure 1: Pipeline diagram of the MapReduce workflow

In the first stage, each review record was read as a JSON object. From each record, the `reviewText` field and the corresponding `category` field were extracted. The preprocessing pipeline consisted of case folding, tokenization into unigrams, stopword removal, and filtering of single-character tokens, following the assignment specification. Tokenization was implemented using a regular-expression-based delimiter set that includes whitespaces, tabs, digits, and the

required punctuation symbols. After preprocessing, duplicate terms within the same review were removed by converting the token list into a set. This ensured that each term contributed at most once per review and category, resulting in document frequencies rather than raw term frequencies.

The first MapReduce job generates the frequency statistics required for the chi-square computation. During the mapper phase, each unique token emits a key-value pair of the form $((\text{term}, \text{category}), 1)$, representing that the term occurs in one review of the given category. In addition, a special key $(\text{"__TOTAL__"}, \text{category})$ is emitted once per review to count the total number of documents in each category. A combiner is used to perform local aggregation of these counts before the shuffle phase, reducing the amount of intermediate data transferred across the cluster. During shuffle and sort, Hadoop groups identical keys together. The reducer then sums all partial counts and produces global document-frequency statistics in the form $((\text{term}, \text{category}), \text{df})$ and category totals in the form $(\text{"__TOTAL__"}, \text{category}), \text{Nc})$.

In the second stage, the output of the first MapReduce job is used directly as the input for a separate Hadoop MapReduce job, `MRj-chi-square.py`. The mapper of this job parses the aggregated counts from the first stage and emits two types of intermediate keys. Category totals are emitted as $((\text{"0_TOTAL"}, \text{category}), \text{Nc})$, while term-category document frequencies are emitted as $((\text{"1_TERM"}, \text{term}), (\text{category}, \text{df}))$. The prefixes `0_TOTAL` and `1_TERM` ensure that category totals are processed before normal term counts during reducer execution.

The second job uses one reducer because the final result requires global category totals, global top-75 term lists per category, and one merged dictionary across all categories. The reducer first stores the category totals and computes the total number of documents N . For each term, it collects the document frequencies across all categories and calculates a chi-square score for every term-category pair. The contingency table values are defined as follows: a is the number of documents in the target category containing the term, b is the number of documents outside the category containing the term, c is the number of documents in the category not containing the term, and d is the number of documents outside the category not containing the term. The chi-square score is then computed using the standard formula:

$$\chi^2 = \frac{N(ad - bc)^2}{(a + b)(c + d)(a + c)(b + d)}$$

To keep the computation memory-efficient, the reducer maintains a bounded min-heap of size 75 for each category. Whenever a new chi-square score is computed, the corresponding term is inserted into the heap only if it belongs to the current top 75 terms for that category. After all terms have been processed, the selected terms are sorted in descending order of chi-square score, with alphabetical ordering used as a tie-breaker. Finally, all selected category-specific term lists are merged into a global dictionary, which is sorted alphabetically as required by the assignment. An example of the generated output for one product category is provided in Appendix A.

To support both development and full-scale execution, separate shell scripts were implemented for the development dataset and the complete dataset. Each script runs the two MapReduce jobs sequentially, stores intermediate outputs in HDFS, and finally merges the result into the local `result/` directory. For evaluation, the final pipeline was executed on the full dataset and completed in approximately 18 minutes. A screenshot of the command-line execution log is included in Appendix B.

4. Conclusion

In this project, we have utilized the MapReduce programming paradigm to evaluate the potential of discriminative unigrams in processing large-scale Amazon review datasets and identifying features in large-scale datasets related to many product categories. Our primary goal was to determine whether certain words can effectively represent strong associations to product categories and thus be beneficial for use in text classification processes. Due to the fact that our dataset included millions of reviews and a large quantity of unformatted text, utilizing a scalable distributed computational model was critical. As part of the preprocessing steps involved in preparing the unformatted textual data for efficient analysis, we performed tokenization, case folding, stopwords elimination and removed single character tokens from the data.

Following the preprocessing steps, the frequency of terms per category was evaluated using the mrjob toolset on Hadoop; subsequent to that evaluation, we computed χ^2 statistics to assess the degree of association between individual terms and categories. We then ranked all terms according to their χ^2 value, selected the top 75 terms most associated with each category, and combined them into a single dictionary representing all product categories. In general, our findings demonstrate that the integration of statistical feature selection methodologies with distributed computing represents an optimal strategy for analyzing large volumes of text-based data. Although the development dataset was used for testing and validation, the same pipeline was developed to be capable of scaling and analyzing the whole dataset. Potential areas of improvement in future iterations of this work could include reducing runtimes further, conducting experiments regarding alternative feature selection methodologies, and expanding our existing approach to support more sophisticated forms of text-based classification.

A. Output example for one product category

This appendix presents an example of the generated output for `Apps_for_Android games` product category. The terms are ranked in descending order according to their chi-square scores.

```
Apps_for_Android games:2537037.1056 play:2247115.1857 graphics:1713265.4482
kindle:1623429.5419 addicting:1242808.2184 addictive:1035936.0442
challenging:992713.5394 coins:955141.6797 fire:873237.7714
playing:858860.1408 puzzles:685415.5497 apps:581592.884
levels:524737.6223 free:501002.7282 download:468874.6801
downloaded:401933.9544 ads:391326.5745 bingo:326777.7663
gameplay:292273.0536 flappy:278066.6912 uninstalled:268092.4271
facebook:244862.8002 mahjong:224833.1723 solitaire:208979.8429
puzzle:202466.7491 played:195723.3961 faotd:192946.7263
waster:191535.2532 deleted:177813.0079 bought:177044.8174
android:167433.6188 minecraft:151811.8195 permissions:150550.721
quality:149208.721 earn:147636.8012 glitches:146220.3342
price:143873.8302 delete:140480.6472 author:135121.9124
made:132578.2202 challenge:132521.7696 reading:132386.7102
addicted:131976.0231 years:129977.8912 fit:125061.1096
freezes:124941.4587 uninstall:115769.5841 unlock:115529.3259
characters:111292.4771 series:110420.0177 downloading:109572.8789
written:106710.5702 multiplayer:103978.0854 hints:94662.7469
size:91927.6947 brain:89846.1004 crashes:88833.9588
logos:88734.4098 bored:88520.2553 appstore:87729.0947
awesome:87075.0368 end:86583.0092 sims:86515.1848
developer:86228.9582 put:86093.6467 small:85847.0008
back:85710.0807 perfect:83950.8371 feel:83251.6801
light:82042.8312 apos:78491.832 temple:73684.3592
tetris:73628.0379 ordered:73215.7105 gameloft:71976.9588
```

B. Full Dataset Execution Log

real	17m43.326s
user	1m16.118s
sys	0m7.223s

Figure 2: Command-line log of the full dataset MapReduce pipeline execution